Problem 1

1. $f(n) = 3n^2 + 4n + 5$ vs. $g(n) = 2n^2 + 6n \log n$

   $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{3n^2+4n+5}{2n^2+6n\log n} = \lim_{n\to\infty} \frac{(n^2)\left(3+\frac{4}{n}+\frac{5}{n^2}\right)}{(n^2)\left(2+6\frac{\log n}{n}\right)} = \lim_{n\to\infty} \frac{3+\frac{4}{n}+\frac{5}{n^2}}{2+6\frac{\log n}{n}}$

   $= \frac{3}{2}$ given $\lim_{n\to\infty} \frac{\log n}{n} = 0$

   By Asymptotic Time Complexity: $f(n) = \theta(g(n))$

2. $f(n) = 2^n$ vs. $g(n) = n^3$

   $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{2^n}{n^3} = \lim_{n\to\infty} \frac{2^n \ln(2)}{3n^2}$ by L'Hospital's rule

   $= \lim_{n\to\infty} \frac{2^n \ln^2(2)}{6n}$ by L'Hospital's rule

   $= \lim_{n\to\infty} \frac{2^n \ln^3(2)}{6} = \infty$

   By Asymptotic Time Complexity: $f(n) = \omega(g(n))$

3. $f(n) = 1000$ vs. $g(n) = \log n$

   $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{1000}{\log n} = 0$

   By Asymptotic Time Complexity: $f(n) = o(g(n))$

4. $f(n) = \log^\gamma n$

vs. $g(n) = 2n$, where ɣ is a constant.

$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{\log^\gamma n}{2n}$ by L'Hospital's rule

$= \lim_{n\to\infty} \frac{\gamma \log^{\gamma-1} n}{2n\ln(2)}$

$= \lim_{n\to\infty} \frac{\gamma!}{\ln^\gamma(2) 2n \ln(2)}$ by $\lceil\gamma\rceil$ times L'Hospital's rule

$= 0$

By Asymptotic Time Complexity: $f(n) = o(g(n))$

5. $f(n) = (n-1)!$ vs. $g(n) = n!$

   $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{(n-1)!}{n!} = \lim_{n\to\infty} \frac{1}{n} = 0$

   By Asymptotic Time Complexity: $f(n) = o(g(n))$

Problem 2 - COUNT3SUM

input: an array of n distinct integer a := $[a_1, ..., a_n]$ and a target natural number c.
output: # of all triples of indices (i,j,k) such that $i < j < k$ and $a_i + a_j + a_k = c$.

Algo:

```
def 3SUM(a, c):
  sort(a)
  cnt=0
  n=len(a)
  for i from 1 to n-2 do
    cnt += 2SUM(a[i+1:n], c-a[i])
  return cnt
```

```
def 2SUM(a, c):
  j=1, k=len(a), cnt=0
  while j<k do
    if a[j]+a[k]==c then
      cnt++
      j++, k--
    else if a[j]+a[k]<c then
      j++
    else
      k--
  return cnt
```

Proof of Correct.: We sort the array in $O(n \log n)$ time and view 3SUM as a series of 2SUM problems. For each element $a[i]$, we fix it as the first element of our triple and search for pairs $(a[j], a[k])$ such that $a[j] + a[k] = c - a[i]$. Since we iterate through all possible $a[i]$ and only look to its right, we ensure $i < j < k$. For the 2SUM step on the sorted subarray, we have two pointers: left at i+1 and right at n and we target $z = c - a[i]$. If there are no $a, b$ such that $a[a] + a[b] = z$, then we won't find them. Suppose $a[a] + a[b] = z$ for some $a \leq b$. Since the algo runs until j=k, there is an iteration such that a=j or b=k. Without loss, suppose that $a = j$ happens first, and $b < k$; the other case is symmetric. As long as $b < k$, we have $a[a] + a[b] < a[j] + a[k]$, and so we will decrease k.
Time Compl.: 2SUM is in $O(n)$ as $k - j$ decrease by one in each iteration, so the algo will stop within $n - 1$ iterations. 3SUM is in $O(n \log n + n^2) = O(n^2)$ as we sort the array in $O(n \log n)$ and then for each of the n elements, we perform a 2SUM operation which takes $O(n)$. Thus, the overall time complexity is $O(n^2)$.

Problem 3: - Recurrence

1. $T(n) = T(n-10) + n$
   I.H.: Assume $T(k) = O(k^2)$ for all $k < n$.
   $T(n) = T(n-10) + n \leq c(n-10)^2 + n$
   I.S.: $\leq cn^2 - 20cn + 100c + n.$
   To complete the induction, we need to show that $-20cn + 100c + n \leq 0$ for sufficiently large n. This can be achieved by choosing $c = 1$ and $n \geq 6$. Therefore, $T(n) = O(n^2)$.

2. $T(n) = 9T\left(\frac{n}{3}\right) + n^2$
   Using Master Theorem, we have $2 = \log_3 9$, so $T(n) = O(n^2 \log n)$

3. $T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{3}\right) + n$
   I.H.: Assume $T(k) = O(k)$ for all $k < n$.
   $T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{3}\right) + n \leq c_1\left(\frac{n}{2}\right) + c_2\left(\frac{n}{3}\right) + n$
   I.S.: $\leq \frac{5}{6}cn + n$ let $c = \max(c_1, c_2)$
   To complete the induction, we need to show that $\frac{5}{6}cn + n \leq cn$. This can be achieved by choosing $c \geq 6$. Therefore, $T(n) = O(n)$.

4. $T(n) = 5T\left(\frac{n}{3}\right) + \log^2 n$
   $5^{\log n}$
   I.H.: Assume $T(k) = O\left(k^{\log 5} - \log^2 k\right)$ for all $k < n$.

$$T(n) = 5T\left(\tfrac{n}{3}\right) + \log^2 n \le 5c\left(\tfrac{n}{3}\right)^{\log 5} - 5c\log^2\left(\tfrac{n}{3}\right) + \log^2 n$$
$$\le cn^{\log 5} - 5c(\log n - \log 3)^2 + \log^2 n$$
$$\le cn^{\log 5} - 5c(\log^2 n - 2\log 3 \log n + \log^2 3) + \log^2 n$$
I.S.: $$\le cn^{\log 5} - (5c-1)\log^2 n + 10c \log 3 \log n - 5c \log^2 3$$

To complete the induction, we need to show that $-(5c-1)\log^2 n + 10c\log 3 \log n - 5c\log^2 3 \le -c\log^2 n$ for sufficiently large n. This can be achieved by choosing $c = 1$ and $n \ge 9$. Therefore, $T(n) = O\left(n^{\log 5} - \log^2 n\right) = O\left(n^{\log 5}\right)$.

5. $T(n) = 2T(n-2) + 1$
   $1 + 2 + ... + 2^k$ where $k = \frac{n}{2}$
   $\left(\frac{1-2^{k+1}}{1-2}\right) = 2^{k+1} - 1 = 2^{\frac{n}{2}+1} - 1$
   I.H.: Assume $T(k) = O\left(2^{\frac{k}{2}} - 1\right)$ for all $k < n$.
   $T(n) = 2T(n-2) + 1 \le 2c2^{\frac{n-2}{2}} - 2c + 1$
   I.S.: $\le c2^{\frac{n}{2}} - 2c + 1$
   To complete the induction, we need to show that $-2c + 1 \le -c$ for sufficiently large n. This can be achieved by choosing $c \ge 1$. Therefore, $T(n) = O\left(2^{\frac{n}{2}} - 1\right) = O\left(2^{\frac{n}{2}}\right)$.

Problem 4: Median of Arrays Two sorted arrays A and B of size n each. Find the median of the combined array of size 2n in O(log n) time.

Algo:

```
def median_of_2arrays(A, B):
  return helper(A, 1, len(A), 1, len(B)-1)
def helper(A, a_start, a_end, B, b_start, b_end):
  n=a_end-a_start+1
  if n==1 then
    return (A[a_start]+B[b_start])/2
  if n==2 then
    return (max(A[a_start], B[b_start]) + min(A[a_end], B[b_end]))/2
  m1_idx=a_start+(n-1)//2
  m2_idx=b_start+(n-1)//2
  m1,m2=A[m1_idx],B[m2_idx]
  if m1==m2 then
    return m1
  if m1<m2 then
    return helper(A, m1_idx, a_end, B, b_start, b_start+(a_end-m1_idx))
  else
    return helper(A, m1_start, a_start+(b_end-m2_idx), B, m2_idx, b_end)
```

Proof of Correct.:
Base cases: for n=1, median of two elements is their average; for n=2, the median is the average of the 2nd and 3rd of the sorted array. Since A and B are sorted, the 2nd and 3rd elements must be among $\{A[a_{\text{start}}], A[a_{\text{end}}], B[b_{\text{start}}], B[b_{\text{end}}]\}$, which are exactly $\max(A[a_{\text{start}}], B[b_{\text{start}}])$ and $\min(A[a_{\text{end}}], B[b_{\text{end}}])$.
Inductive step: For arrays of size n, let $m_1$ and $m_2$ be the medians of A and B. If $m_1 = m_2$, then there are exactly as many elements across both arrays smaller than $m_1$ as there are elements larger than $m_1$. Without loss, suppose $m_1 < m_2$, the overall median t must lie in the range $[m_1, m_2]$; the other case is symmetric where range is $[m_2, m_1]$. Any element in the first half of A and in the second half of B is out of range, so we can discard them safely. As we have removed an equal number of elements from both ends of the sorted sequence, the relative position of the median remains unchanged. By the I.S., the recursion on the remaining sub-arrays of equal size will return

the correct median.
Time Compl.:
The algo find medians in $O(1)$ and discards half of each array, reducing problem size to n/2.

$$T(n) = T(\tfrac{n}{2}) + O(1) \leq c_1 \log(\tfrac{n}{2}) + c_2$$

I.H.: $T(k) = O(\log k)$ for all $k < n$. I.S.:

$$\leq c_1 \log n - c_1 \log 2 + c_2$$

To complete the induction, we need to show that $c_1 \log 2 - c_2 \geq 0$. This can be achieved by choosing $c_1 = 1$ and $c_2 = 0$. Therefore, $T(n) = O(\log n)$.

Problem 5: Maximum Ad Space n consecutive buildings with positive integer height $h_1, ..., h_n \in [1, n^{100}]$ and unit width 1. Find the maximum area of a rectangle that can be formed by choosing a contiguous set of buildings. Find O(n log n) algo for $1 \leq i \leq j \leq n$ to maximize $(j - i + 1) \min_{i \leq k \leq j} h_k$.

Algo:

```
def max_area(h, low, high):
  n_sub=high-low+1
  if n_sub==1 then
    return h[low]
  if n_sub==2 then
    return max(h[low], h[high], 2*min(h[low], h[high]))
  mid=(low+high)//2
  l=max_area(h, low, mid)
  r=max_area(h, mid+1, high)
  c=max_area_cross(h, low, mid, high)
  return max(l, r, c)

def max_area_cross(h, low, mid, high):
  left=mid, right=mid+1
  min_height=min(h[left], h[right])
  max_area=2*min_height
  while left>low or right<high do
    if left==low then
      right+=1
    else if right==high then
      left-=1
    else if h[left-1]>h[right+1] then
      left-=1
    else
      right+=1

    min_height=min(min_height, h[left], h[right])
    area=(right-left+1)*min_height
    max_area=max(max_area, area)

  return max_area
```

Proof of Correct.: Base cases: for n=1, the max space is its height h[low]; for n=2, there are only 3 possible rectangular config and the algo evaulate all. Inductive step: assume the algo finds the max rectangular area for any array of size less than n. For an array of size n, the optimal rectangle spanning from index I to J must fall into below cases: in the left half ($I \leq J \leq$ `mid`), in the right half (`mid` $< I \leq J$), and crosses the midpoint ($I \leq$ `mid` $<$ `mid` $+ 1 \leq J$). Let the optimal crossing rectangle span from I to J with height $H = \min_{k=I}^{J} h[k]$. We prove our points left and right will expand to exactly match $[I, J]$. Without loss, suppose one pointer reaches its optimal boundary first, say left=I while right< J; the other case is symmetric. The algo compares the adjacent elements

$h[I-1]$ and $h[\text{right}+1]$: $h[\text{right}+1] \geq H$, because index right+1 lies inside the optimal window $[I, J]$; $h[I-1] < H$, otherwise the window $[I-1, J]$ would have a height of H and a larger area, contradicting the optimality of $[I, J]$. Since $h[\text{right}+1] \geq H \geq h[I-1]$, the greedy condition correctly chooses to expand right towards J. By symmetry, the pointers are to encompass the optimal window $[I, J]$ and record its max area.
Time Compl.:
max_area_cross is in $O(n)$ as either left decrease or right increases by one in each iteration, so the loop will stop within n-2 iterations.

$$\begin{aligned} T(n) = 2T(\tfrac{n}{2}) + O(n) &\leq 2c_1 \tfrac{n}{2} \log \tfrac{n}{2} + c_2 n \\ &\leq c_1 n(\log n - \log 2) + c_2 n \\ &\leq c_1 n \log n - (c_1 \log 2 - c_2) n \end{aligned}$$

I.H.: $T(n) = O(n \log n)$. I.S.:
To complete the induction, we need to show that $c_1 \log 2 - c_2 \geq 0$ for suffiently large n. This can be achieved by choosing $c_1 = 1, c_2 = 1$. Therefore, $T(n) = O(n \log n)$